

SelVeri: Self-Verifying Programming Language
High Level Language Syntax and Semantics Document
v0.1

Yusuf Demir
Yağız Kaan Aydoğdu

February 26, 2026

1 Syntax

1.1 Lexical Categories

Identities (names) of objects must start with a letter or underscore, and continue with letters, digits or undercases. Literals are integers or floating point numbers as SelVeri supports just these types.

$$\begin{aligned}\langle \textit{Ident} \rangle &::= [\text{A-Za-z_}][\text{A-Za-z0-9_}]^* \\ \langle \textit{IntLit} \rangle &::= [0-9]^+ \\ \langle \textit{FloatLit} \rangle &::= [0-9]^+ \setminus \cdot [0-9]^+\end{aligned}$$

1.2 Types

All variables are of types of **Int**, **Float** or **List** of those. Lists contain values of only one type and their size **must** be explicitly stated in the declaration as an arithmetic expression.

$$\langle \textit{Type} \rangle ::= \text{Int} \mid \text{Float} \mid \text{List}[\langle \textit{Type} \rangle, \langle \textit{AExp} \rangle]$$

1.3 Immediates

$$\langle \textit{Imm} \rangle ::= \langle \textit{IntLit} \rangle \mid \langle \textit{FloatLit} \rangle \mid [] \mid [\langle \textit{Imm} \rangle (, \langle \textit{Imm} \rangle)^*]$$

1.4 Expressions

Arithmetic Expressions ($\textit{AExp}$)

$$\begin{aligned}\langle \textit{AExp} \rangle &::= \langle \textit{Imm} \rangle \mid \langle \textit{Ident} \rangle \mid (\langle \textit{AExp} \rangle) \\ &\mid \langle \textit{AExp} \rangle + \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle - \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle * \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle / \langle \textit{AExp} \rangle \\ &\mid \text{len}(\langle \textit{Ident} \rangle) \mid \langle \textit{Ident} \rangle[\langle \textit{AExp} \rangle]\end{aligned}$$

Boolean Expressions ($\textit{BExp}$).

$$\begin{aligned}\langle \textit{BExp} \rangle &::= \text{true} \mid \text{false} \mid \langle \textit{AExp} \rangle \\ &\mid \langle \textit{AExp} \rangle = \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle \leq \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle < \langle \textit{AExp} \rangle \\ &\mid \langle \textit{AExp} \rangle \geq \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle > \langle \textit{AExp} \rangle \\ &\mid \text{not } \langle \textit{BExp} \rangle \mid \langle \textit{BExp} \rangle \text{ and } \langle \textit{BExp} \rangle \mid \langle \textit{BExp} \rangle \text{ or } \langle \textit{BExp} \rangle \mid (\langle \textit{BExp} \rangle)\end{aligned}$$

Expressions

$$\langle \textit{Expr} \rangle ::= \langle \textit{AExpr} \rangle \mid \langle \textit{BExpr} \rangle$$

1.5 Specifications (Assertions / Contracts)

$$\begin{aligned}\langle \textit{Spec} \rangle &::= \langle \textit{FOL} \rangle \mid \langle \textit{PastLTL} \rangle \mid \langle \textit{FutureLTL} \rangle \\ \langle \textit{PastLTL} \rangle &::= \text{Previously } \langle \textit{Spec} \rangle \mid \text{Once } \langle \textit{Spec} \rangle \mid \text{Historically } \langle \textit{Spec} \rangle \mid \langle \textit{Spec} \rangle \text{ Since } \langle \textit{Spec} \rangle \\ \langle \textit{FutureLTL} \rangle &::= \text{Next } \langle \textit{Spec} \rangle \mid \text{Eventually } \langle \textit{Spec} \rangle \mid \text{Always } \langle \textit{Spec} \rangle \mid \langle \textit{Spec} \rangle \text{ Until } \langle \textit{Spec} \rangle \\ \langle \textit{FOL} \rangle &::= \langle \textit{BExp} \rangle \mid \langle \textit{FOL} \rangle \&\& \langle \textit{FOL} \rangle \mid \langle \textit{FOL} \rangle \parallel \langle \textit{FOL} \rangle \\ &\mid ! \langle \textit{FOL} \rangle \mid \langle \textit{FOL} \rangle \Rightarrow \langle \textit{FOL} \rangle \mid (\langle \textit{FOL} \rangle) \\ &\mid \text{Forall } \langle \textit{Ident} \rangle \langle \textit{DomainOpt} \rangle . \langle \textit{FOL} \rangle \\ &\mid \text{Exists } \langle \textit{Ident} \rangle \langle \textit{DomainOpt} \rangle . \langle \textit{FOL} \rangle \\ \langle \textit{DomainOpt} \rangle &::= \epsilon \mid \text{in } \langle \textit{Domain} \rangle \\ \langle \textit{Domain} \rangle &::= \langle \textit{Interval} \rangle \mid \langle \textit{Ident} \rangle \mid \langle \textit{SetLiteral} \rangle \mid \langle \textit{Type} \rangle \\ \langle \textit{Interval} \rangle &::= [\langle \textit{AExp} \rangle, \langle \textit{AExp} \rangle) \mid [\langle \textit{AExp} \rangle, \langle \textit{AExp} \rangle] \\ \langle \textit{SetLiteral} \rangle &::= \{ \langle \textit{AExpList} \rangle \} \\ \langle \textit{AExpList} \rangle &::= \langle \textit{AExp} \rangle \mid \langle \textit{AExp} \rangle, \langle \textit{AExpList} \rangle\end{aligned}$$

1.6 Statements and Programs

Statements

$$\begin{aligned}\langle Decl \rangle &::= \langle Ident \rangle : \langle Type \rangle \\ \langle Assign \rangle &::= \langle Ident \rangle := \langle Expr \rangle \\ \langle ListAssign \rangle &::= \langle Ident \rangle [\langle AExpr \rangle] := \langle Expr \rangle \\ \langle Stmt \rangle &::= \langle Decl \rangle \mid \langle Assign \rangle \mid \langle ListAssign \rangle \mid \text{noop} \\ &\quad \mid \langle Stmt \rangle ; \langle Stmt \rangle \\ &\quad \mid \text{if } \langle BExp \rangle \text{ then } \langle Stmt \rangle [\text{else } \langle Stmt \rangle] \text{ fi} \\ &\quad \mid \text{while } \langle BExp \rangle \text{ do } \langle Stmt \rangle \text{ od} \\ &\quad \mid \{ \langle StmtSeq \rangle \} \\ \langle StmtSeq \rangle &::= \epsilon \mid \langle Contract \rangle (; \langle Contract \rangle)^*\end{aligned}$$

Contracts (inline verification annotations).

$$\langle Contract \rangle ::= \langle Stmt \rangle [\{ \langle FOL \rangle \}]$$

Programs

$$\langle Program \rangle ::= \langle StmtSeq \rangle$$

2 Semantics

2.1 Important notions for formalizing SelVeri

2.1.1 Types

Let **Type** be the set of all possible types. For the current implementation of **SelVeri**, an element of **Type** can be either **Int**, **Float** or a **List** of any dimension, including one of the former two, which can be interpreted as a dependent type. More formally:

- **Int** : **Type**
- **Float** : **Type**
- **List** : $(\Pi t : \mathbf{Type} . (\Pi n : \mathbb{N}^+ . \mathbf{Type}))$

2.1.2 Immediates

Let **Imm** be the set of syntactic immediate values supported by **SelVeri**, e.g. 5 or 3.14. For each type, we need an evaluation function that translates syntactic immediates into the corresponding semantic values.

$$\mathcal{I} : \mathbf{Imm} \cap \mathbf{Int} \rightarrow \mathbb{Z}$$

$$\mathcal{F} : \mathbf{Imm} \cap \mathbf{Float} \rightarrow \mathbb{R}$$

$$\mathcal{L}_{\mathbf{List}[t, n]} : \mathbf{Imm} \cap \mathbf{List}[t, n] \rightarrow A^n$$

where A is the codomain of $\mathcal{L}_t$. The reader should notice that:

- $\mathcal{L}_{\mathbf{Int}} = \mathcal{I}$
- $\mathcal{L}_{\mathbf{Float}} = \mathcal{F}$

For simplicity, we also define $\mathcal{L} := \bigcup_{t \in \mathbf{Type}} \mathcal{L}_t$

2.1.3 Scope

Let **Scope** be the set of all scopes and let **Var** be the infinite set of all syntactic variables. Each scope is a representation of 'known' variables in a given block of statements including the variables declared until program execution reaches the corresponding block. Formally,

$$\forall s \in \mathbf{Scope} . s : \mathbf{Var} \rightarrow \mathbf{Type}$$

Initially, each scope starts its lifetime as an empty scope, denoted as $\tilde{s}$,

$$\forall x \in \mathbf{Var} . \tilde{s}(x) = \mathbf{Void}$$

where **Void** is the empty type.

Moreover, each scope comes along with a (possibly empty) **parent_scope**, which is the scope of the program block that 'called' the current block. In the **SelVeri** implementation, a variable is searched in the **parent_scope**, if it cannot be found in the current scope.

2.1.4 Program states

Let **State** be the set of all possible program states and **Val**¹ be the set of all semantic values. Each state is in fact a map from **Var** and **Scope** to **Val**, representing the value of a given variable in the specified scope. Formally,

$$\forall \sigma \in \mathbf{State}. \sigma : \mathbf{Var} \times \mathbf{Scope} \rightarrow \mathbf{Val}$$

Each program starts with an empty state, $\tilde{\sigma}$ in which each declared variable has the default value of this type, e.g. 0 for **Int**, 0.0 for **Float** and [0, 0, 0] for **List[Int, 3]**.

Additionally, each program state carries an **error_flag** that is set if and only if an error occurs during the execution of the program. A cause for this can be using a variable before declaring it. In order to embed this misuse as an error into our semantics, we add the following rule to our program states:

$$\forall \sigma \in \mathbf{State}. \sigma(x, s) = \perp \quad \text{if } s(x) = \mathbf{Void}.$$

Before moving to the next section, reader should be aware of an abuse of notation we use for convenient representation of erroneous states:

- σ represents a program state without any errors.
- $\hat{\sigma}$ represents a program state with an error.
- $\underline{\sigma}$ represents any program state (may be error-free or erroneous).

¹The reader can notice that $\text{ran}\mathcal{I}$, $\text{ran}\mathcal{F}$ and $\text{ran}\mathcal{L}$ are all subsets of **Val**.

2.2 Semantics of arithmetic expressions

Let **AExp** be the set of all arithmetic expressions. We define the arithmetic valuation function $\mathcal{A}$, representing the semantics of the arithmetic expressions in **SelVeri**.

$$\mathcal{A} : \mathbf{AExp} \times \mathbf{State} \times \mathbf{Scope} \rightarrow \mathbf{Val}$$

$\mathcal{A}(x, \sigma, s) = \mathcal{I}(x)$	if $x \in \mathbf{Imm} \wedge \mathcal{T}(x, s) = \mathbf{Int}$
$\mathcal{A}(x, \sigma, s) = \mathcal{F}(x)$	if $x \in \mathbf{Imm} \wedge \mathcal{T}(x, s) = \mathbf{Float}$
$\mathcal{A}(x, \sigma, s) = \mathcal{L}_{\mathbf{List}[t, n]}(x)$	if $x \in \mathbf{Imm} \wedge \exists t \in \mathbf{Type} \cdot s(x) = \mathbf{List}[t] \wedge \mathcal{A}(\mathbf{len}(x), \sigma, s) = n$
$\mathcal{A}(x, \sigma, s) = \sigma(x, s)$	if $x \in \mathbf{Var}$
$\mathcal{A}(a_1 + a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) + \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$	
$\mathcal{A}(a_1 * a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) \cdot \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$	
$\mathcal{A}(a_1 - a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) - \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s) \neq \perp \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$	
$\mathcal{A}(a_1 / a_2, \sigma, s) = \begin{cases} \lfloor \mathcal{A}(a_1, \sigma, s) / \mathcal{A}(a_2, \sigma, s) \rfloor & \text{if } \mathcal{A}(a_2, \sigma, s) \neq 0 \wedge (\mathcal{A}(a_1, \sigma, s) \neq \perp \wedge \mathcal{A}(a_2, \sigma, s) \neq \perp) \\ \perp & \text{if } \mathcal{A}(a_2, \sigma, s) = 0 \vee (\mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp) \end{cases}$	

Table 1: The semantics of arithmetic expressions

2.3 Semantics of boolean expressions

Let **BExp** be the set of all boolean expressions. We define the arithmetic valuation function $\mathcal{B}$, representing the semantics of the boolean expressions in **SelVeri**.

$$\mathcal{B} : \mathbf{BExp} \times \mathbf{State} \times \mathbf{Scope} \rightarrow \{0, 1, \perp\}$$

$\mathcal{B}(\text{true}, \sigma, s)$	$= 1$
$\mathcal{B}(\text{false}, \sigma, s)$	$= 0$
$\mathcal{B}(a, \sigma, s)$ ²	$= \begin{cases} 1 & \text{if } \mathcal{A}(a, \sigma, s) \neq 0 \\ 0 & \text{if } \mathcal{A}(a, \sigma, s) = 0 \\ \perp & \text{if } \mathcal{A}(a, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(a_1 = a_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \mathcal{A}(a_2, \sigma, s) \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(a_1 \leq a_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) \leq \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) > \mathcal{A}(a_2, \sigma, s) \\ \perp & \text{if } \mathcal{A}(a_1, \sigma, s) = \perp \vee \mathcal{A}(a_2, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(\text{not } b, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{B}(b, \sigma, s) = 0 \\ 0 & \text{if } \mathcal{B}(b, \sigma, s) = 1 \\ \perp & \text{if } \mathcal{B}(b, \sigma, s) = \perp \end{cases}$
$\mathcal{B}(b_1 \text{ and } b_2, \sigma, s)$	$= \begin{cases} 1 & \text{if } \mathcal{B}(b_1, \sigma, s) = 1 \text{ and } \mathcal{B}(b_2, \sigma, s) = 1 \\ 0 & \text{if } \mathcal{B}(b_1, \sigma, s) = 0 \text{ or } \mathcal{B}(b_2, \sigma, s) = 0 \\ \perp & \text{if } \mathcal{B}(b_1, \sigma, s) = \perp \text{ or } \mathcal{B}(b_2, \sigma, s) = \perp \end{cases}$

Table 2: Semantics of boolean expressions

²To have this valuation rule, we need $\mathbf{AExp} \subseteq \mathbf{BExp}$ which is the case for **SelVeri**. What this rule means is, every arithmetic expression can also be interpreted as a boolean expression depending on the context and its truth value is determined by an equality to zero check, similar to the C programming language.

2.4 Structural operational semantics of the statements

$[\text{decl}_{\text{os}}]$	$\langle x : t, \sigma, s \rangle \rightarrow s[x \mapsto t] \quad \text{if } t \in \mathbf{Type} \wedge s(x) = \mathbf{Void}$
$[\text{decl}_{\text{os}}^\perp]$	$\langle x : t, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } t \notin \mathbf{Type} \vee s(x) \neq \mathbf{Void}$
$[\text{ass}_{\text{os}}]$	$\langle x := a, \sigma, s \rangle \rightarrow \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad \text{if } \mathcal{A}(a, \sigma, s) \neq \perp$
$[\text{ass}_{\text{os}}^\perp]$	$\langle x := a, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } \mathcal{A}(a, \sigma, s) = \perp$
$[\text{noop}_{\text{os}}]$	$\langle \mathbf{noop}, \sigma, s \rangle \rightarrow \sigma$
$[\text{comp}_{\text{os}}^1]$	$\frac{\langle S_1, \sigma, s \rangle \rightarrow \langle S'_1, \underline{\sigma}', s' \rangle}{\langle S_1; S_2, \sigma, s \rangle \rightarrow \langle S'_1; S_2, \underline{\sigma}', s' \rangle}$
$[\text{comp}_{\text{os}}^2]$	$\frac{\langle S_1, \sigma, s \rangle \rightarrow \underline{\sigma}'}{\langle S_1; S_2, \sigma, s \rangle \rightarrow \langle S_2, \underline{\sigma}', s \rangle}$
$[\text{if}_{\text{os}}^1]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}^3, \sigma, s \rangle \rightarrow \langle S_1, \sigma, s \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{if}_{\text{os}}^0]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \rightarrow \langle S_2, \sigma, s \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{if}_{\text{os}}^\perp]$	$\langle \text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma, s \rangle \rightarrow \langle S_1, \sigma, s \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{while}_{\text{os}}^1]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \langle S; \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{while}_{\text{os}}^0]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \sigma \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{while}_{\text{os}}^\perp]$	$\langle \text{while } b \text{ do } S \text{ od}, \sigma, s \rangle \rightarrow \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{err}_{\text{os}}]$	$\langle S, \hat{\sigma}, s \rangle \rightarrow \hat{\sigma}$

Table 3: Small-step operational semantics for statements

³For each if-rule, an if-statement without the **else**-part, namely a statement of the form "if b then S fi" can be treated as "if b then S else **noop** fi"